BIRZEIT UNIVERSITY

Faculty of Engineering & Technology Electrical & Computer Engineering Department

Interfacing techniques – ENCS4380

Arduino Homework

Prepared by: Ruba Aldaghamin

Instructor: Dr. Wasel Ghanem

Section: 2

Date: 2025/12/27

I. Question 1: Hardware & Pin Change Interrupts

1. Circuit Design

The goal of this task is to control two LEDs using interrupt-driven programming without using the `attachInterrupt()` function or polling loops. The system must respond immediately to button presses using direct register manipulation on the ATmega328P.

- **Button A (INT0):** Toggles LED 1 on a **RISING** edge (Press).
- **Button B (PCINT0):** Toggles LED 2 on **ANY CHANGE** (Press or Release).

Link:
https://www.tinkercad.com/things/gnTnIOo9Wds-question1?sharecode=6QR5PFXgeJTD_5_iiurZf3THDM9mUpVZgJQK06RuyEo

2. Hardware Configuration

The circuit was constructed using the "Active High" configuration, where buttons are connected to 5V and use 10kohm pull-down resistors to Ground.

Component	***Arduino Pin***	***ATmega Port***	***Connection Detail***
Button A	*Digital Pin 2*	*PD2 (INT0)*	*Connected to 5V; 10kohm pull-down to GND.*
Button B	*Digital Pin 8*	*PB0 (PCINT0)*	*Connected to 5V; 10kohm pull-down to GND.*
LED 1	*Digital Pin 13*	*PB5*	*Anode to Pin 13; Cathode to GND via 220ohm resistor.*
LED 2	*Digital Pin 12*	*PB4*	*Anode to Pin 12; Cathode to GND via 220ohm resistor.*

3. Software Implementation

The following C program configures the **EICRA** register for the External Interrupt and the **PCICR/PCMSK0** registers for the Pin Change Interrupt.

```
#include <avr/io.h>
#include <avr/interrupt.h>

void setup() {
  DDRB |= (1 << DDB5) | (1 << DDB4)
  DDRD &= ~(1 << DDD2);
  DDRB &= ~(1 << DDB0);


  EICRA |= (1 << ISC01) | (1 << ISC00);

  EIMSK |= (1 << INT0);
  PCICR |= (1 << PCIE0);

  PCMSK0 |= (1 << PCINT0);

  sei();
}

void loop() {
}
ISR(INT0_vect) {
  PORTB ^= (1 << PORTB5);
}

ISR(PCINT0_vect) {
  PORTB ^= (1 << PORTB4);
}
```

4. Theory of operation

Explaining the INT0 Configuration (Button A)

The Concept:

We want the LED to toggle only when you press the button down, not when you release it. This requires detecting a specific electrical change called a Rising Edge (voltage going from 0V to 5V).

```
EICRA |= (1 << ISC01) | (1 << ISC00);
```

- EICRA (External Interrupt Control Register A): This register controls *how* the interrupt triggers.
- ISC01 and ISC00: These are the "Sense Control" bits for INT0.
- The Logic:
    - If you set both bits to 0, it triggers on "Low Level".
    - If you set them to 1 and 0, it triggers on "Falling Edge".
    - By setting both to 1 (Binary 11), we explicitly tell the hardware: *"Ignore everything except the exact moment the voltage jumps from Low to High."* This is why the LED doesn't change when you let go of the button.

```
EIMSK |= (1 << INT0);
```

Even if you configure the edge correctly, the interrupt is "off" by default. EIMSK (External Interrupt Mask Register) is the specific On/Off switch for the INT0 pin. Setting this bit effectively "unmutes" the pin so the processor listens to it.

Explaining the PCINT0 Configuration (Button B)

The Concept:

We want the LED to turn ON when pressed and OFF when released. Since standard interrupts (like INT0) are usually configured for one edge, we use a Pin Change Interrupt. This type of interrupt is designed to trigger on Any Change (both 0V to 5V AND 5V to 0V).

```
PCICR |= (1 << PCIE0);
```

- The "Group" Switch: Pin Change Interrupts are grouped by "Ports". All pins on Port B (Pins 8 to 13) share one single interrupt line called PCIE0.
- This line of code turns on the "master switch" for the entire Port B group. Without this, no pin on Port B can trigger an interrupt.

```
PCMSK0 |= (1 << PCINT0);
```

- The "Specific" Filter: Since we turned on the whole group (Port B) above, *any* noisy pin on Port B could theoretically trigger the code.
- PCMSK0 (Pin Change Mask Register 0): This register lets us pick *exactly which pin* in the group we care about.
- By setting the PCINT0 bit (which corresponds to Pin 8), we tell the processor: *"I know I enabled the whole group, but only wake me up if Pin 8 changes."*

ISRs (Interrupt Service Routines

```
ISR(INT0_vect) { ... }

ISR(PCINT0_vect) { ... }
```

- Hardware Function Call: Normally, functions only run when you call them in main(). An ISR is special—it is never called by your code. It is called directly by the hardware.
- Preemption: When the button is pressed, the processor immediately pauses whatever it was doing (even if it was in the middle of a calculation), jumps to this { ... } block, executes the toggle command, and then goes back to exactly where it left off.
- The Logic inside:
    - PORTB ^= (1 << PORTB5); uses the XOR operator (^).
    - If the bit was 1, 1 ^ 1 = 0 (Turns Off).
    - If the bit was 0, 0 ^ 1 = 1 (Turns On).

5. Simulation

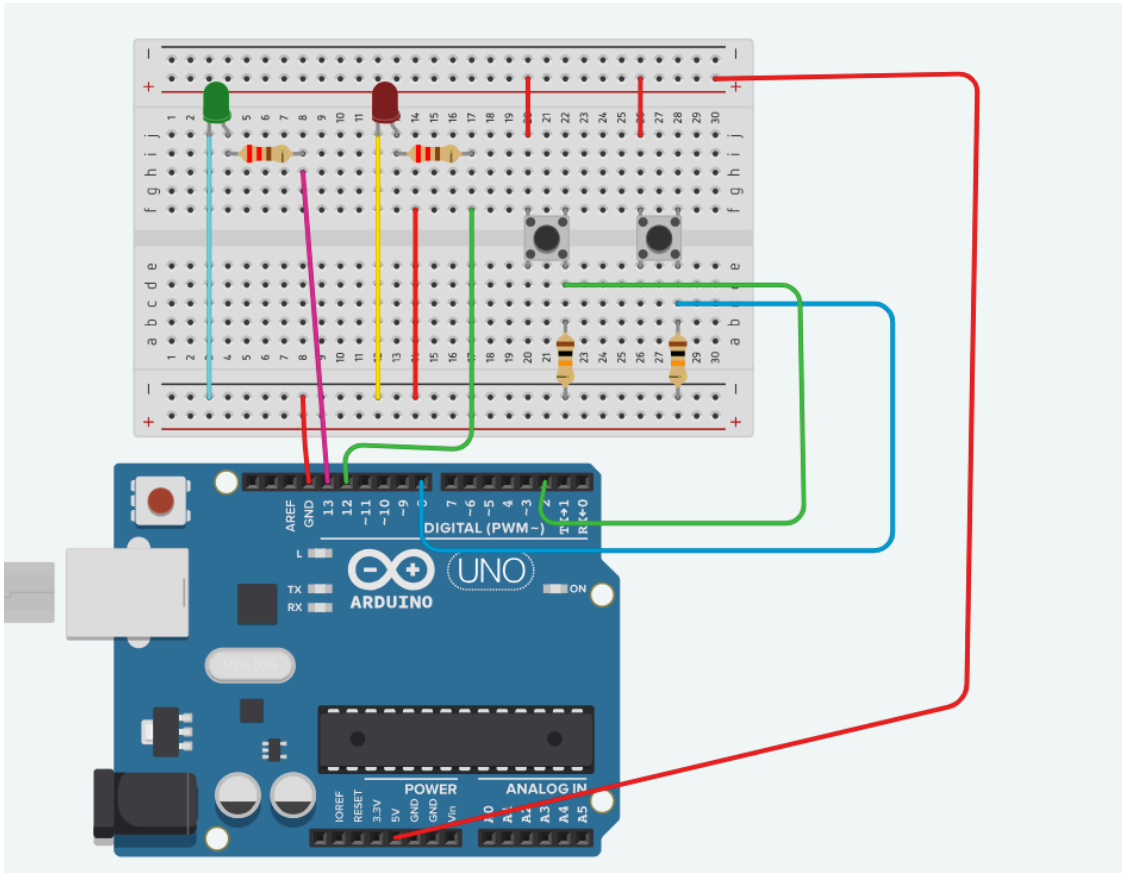


Figure 1 : question 1 mode 0

- **LED 1 (Controlled by Button A):** When Button A was pressed, LED 1 toggled its state (turning ON or OFF) and remained in that state after the button was released. This confirms the **INT0** configuration for a **Rising Edge** trigger; the interrupt service routine executed only once during the initial press (Logic 0 to 1), effectively creating a "latching" switch.
- **LED 2 (Controlled by Button B):** When Button B was pressed and held, LED 2 turned ON, and immediately turned OFF upon release. This confirms the **PCINT0** configuration for **Any Change** trigger. The interrupt executed once on the press (toggling the LED ON) and a second time on the release (toggling the LED OFF), creating a "momentary" switch behavior.

Question 2: Software Interrupts using Timer1 (CTC Mode)

1. Circuit Design

- **Circuit Description:The goal of this task is to generate a precise 1 Hz timing event (blinking an LED every 1 second) using the 16-bit Timer1 . Instead of using `delay()`, the program must use CTC (Clear Timer on Compare Match) Mode to trigger an interrupt exactly every 500ms (0.5 seconds), toggling the LED to achieve a full 1 Hz cycle (On-Off)**
    - **TinkerCad Simulation Link:** https://www.tinkercad.com/things/gpsuJ8q1Wj6-question2?sharecode=undefined

2. Hardware Configuration

For this task, a single LED is controlled by Pin 9, which corresponds to Port B, Pin 1 (PB1) on the ATmega328P.

Component	Arduino Pin	ATmega Port	Connection Detail
LED	Digital Pin 9	PB1	Anode to Pin 9; Cathode to GND via 220OHM resistor.

3. Software Implementation

```
#include <avr/io.h>
#include <avr/interrupt.h>

void setup() {
  DDRB |= (1 << DDB1);
  TCCR1A = 0;
  TCCR1B = 0;
  TCCR1B |= (1 << WGM12);

  TCCR1B |= (1 << CS12);

  OCR1A = 31249;
  TIMSK1 |= (1 << OCIE1A);

  sei();
}

void loop() {
}

ISR(TIMER1_COMPA_vect) {
  PORTB ^= (1 << PORTB1);
}
```

4. Theory of Operation

The precise timing is achieved by manipulating specific register bits to match the 16MHz system clock to our desired frequency.

1. Setting CTC Mode:

TCCR1B |= (1 << WGM12);

Standard timers count up until they overflow (reach the maximum value). **CTC Mode** changes this behavior. It tells the timer to count up to the value stored in `OCR1A` and then immediately **reset to 0** automatically. This creates a perfectly repetitive cycle without requiring manual resets in the code.

2. Calculating the Target Count (OCR1A):

```
OCR1A = 31249;
```

This number dictates the speed. Using the formula provided in the requirements:

$$\text{Target}= \frac{Clock\ Speed(Hz)}{Prescaler \times Desired\ Frequency(Hz)} - 1$$

$$\text{Target}= \frac{16{,}000{,}000}{256 \times 2} - 1 = 31249$$

- **Prescaler 256:** Slows the 16MHz clock down to 62,500 ticks per second.
- **Frequency 2 Hz:** We calculate for 2 Hz (0.5 seconds) because a full "Blink" consists of two phases: ON (0.5s) and OFF (0.5s).
- The result is that the interrupt triggers exactly every 500ms.

3. Enabling the Interrupt:

```
TIMSK1 |= (1 << OCIE1A);
```

This enables the **Output Compare A Match Interrupt**. Without this line, the timer would count and reset silently in the background, but the processor would never pause to run our `ISR` code (which toggles the LED)

5. Simulation

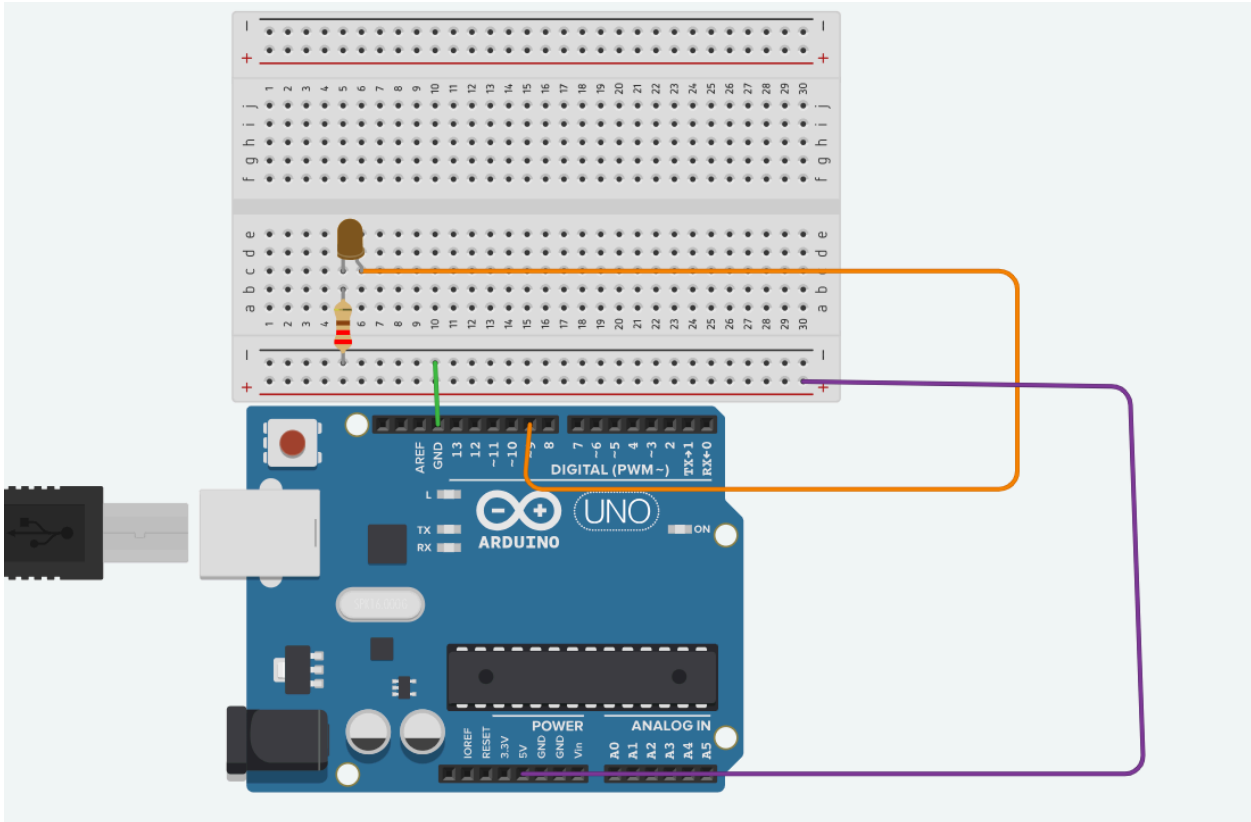


Figure 3 : question 2 implementation

Upon running the simulation, the LED connected to Pin 9 demonstrated a steady, rhythmic blinking pattern. The LED remained ON for exactly 0.5 seconds and OFF for 0.5 seconds, resulting in a total period of 1 second (1 Hz frequency). This confirms that Timer1 was correctly counting to 31,249 ticks before triggering the interrupt and toggling the pin state

Question 3: Software Interrupts using Timer2 (Overflow Mode)

1. Circuit Design

The goal of this task is to use the **8-bit Timer2** to handle timing tasks via overflow interrupts. Since an 8-bit timer cannot count up to 1 second directly (it overflows too quickly), the program must use a **Prescaler of 1024** and a software counter variable to count multiple overflows until the total time reaches approximately 1 second .

- **TinkerCad Simulation Link :**

  https://www.tinkercad.com/things/l3fMdZDAl14-question3

2. Hardware Configuration

The LED for this task is connected to the onboard LED pin, which is Pin 13 (Port B, Pin 5).

Component	***Arduino Pin***	***ATmega Port***	***Connection Detail***
LED	*Digital Pin 13*	*PB5*	*Anode to Pin 13; Cathode to GND via 220ohm resistor.*

3. Software Implementation

```
#include <avr/io.h>
#include <avr/interrupt.h>

volatile int overflowCount = 0;

void setup() {
  DDRB |= (1 << DDB5);

  TCCR2A = 0;
  TCCR2B = 0;

  TCCR2B |= (1 << CS22) | (1 << CS21) | (1 << CS20);
  TIMSK2 |= (1 << TOIE2);

  TCNT2 = 0;
  sei();
}

void loop() {
}

ISR(TIMER2_OVF_vect) {
  overflowCount++;
  if (overflowCount >= 61) {
    PORTB ^= (1 << PORTB5);
    overflowCount = 0;
  }
}
```

4. Theory of Operation

This method relies on "nesting" a hardware timer inside a software counter to achieve longer durations than the hardware supports natively.

1. The Overflow Limitation:

Timer2 is an 8-bit timer, meaning it can only count from 0 to 255. Even with the maximum Prescaler (1024), the timer fills up very quickly.

Time_overflowt= $\frac{1}{16{,}000{,}000}$ (24) (256) =16.384ms

Since 16ms is much faster than the 1 second blinking goal, we cannot just toggle the LED every time the interrupt fires.

2. The Software Counter Strategy:

To bridge the gap between 16ms and 1000ms (1 second), we use a variable overflowCount inside

the ISR.

Required overflows= $\frac{1000ms}{16.384ms}$ = 61.03

The ISR runs every 16ms, but it only performs the "Toggle Action" when the counter hits 61. This effectively multiplies the timer's duration by 61.

3. volatile Variable:

5. Simulation

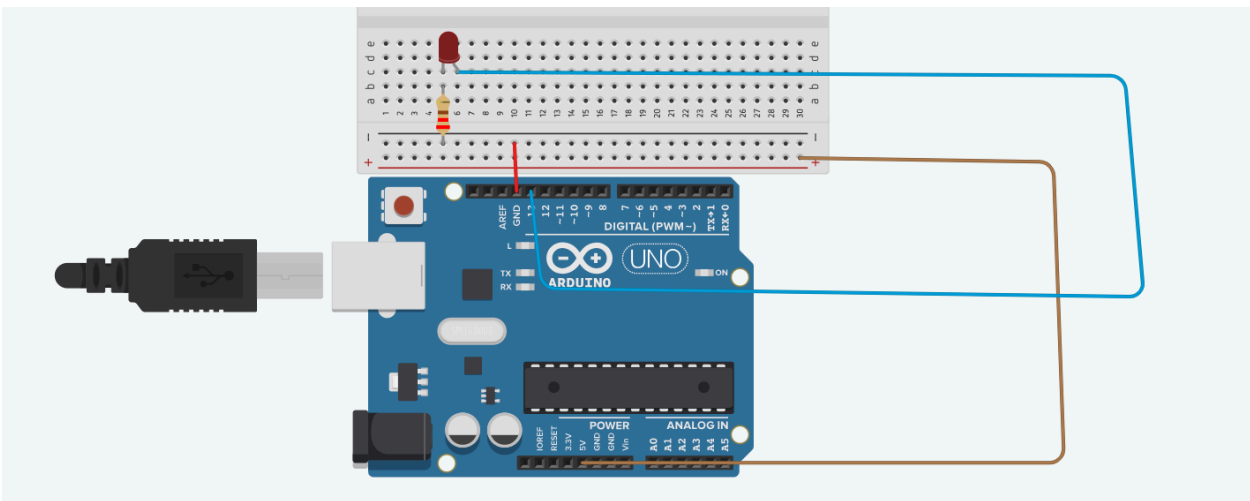


Figure 4 : question 3 implementation

Upon running the simulation, the LED on Pin 13 remained static for the first 60 interrupt cycles (approximately 984ms). On the 61st cycle, the LED toggled state. This resulted in a perceptible blink rate of exactly 1 Hz, confirming that the software counter successfully extended the 8-bit timer's short overflow period into a 1-second delay.

V. Question 4: LCD in 4-bit Mode (Custom Driver)

1. **Circuit Design**
   The goal of this task is to manually configure the 8-bit Timer2 to generate Pulse Width Modulation (PWM) signals without using analogWrite(). The objective is to drive two LEDs with different duty cycles (brightness levels) by manipulating the Wave Generation Mode (WGM) and Compare Output Mode (COM) bits directly.

   - **TinkerCad Simulation Link:** Circuit design task4 - Tinkercad

2. Hardware Configuration

The circuit requires connecting LEDs to the specific Output Compare pins for Timer2 (OC2A and OC2B).

Component	Arduino Pin	ATmega Port	Connection Detail
LED A	Digital Pin 11	PB3 (OC2A)	Anode to Pin 11; Cathode to GND via 220ohm resistor.
LED B	Digital Pin 3	PD3 (OC2B)	Anode to Pin 3; Cathode to GND via 220ohm resistor.

3. Software Implementation

The following C program configures **TCCR2A** for Fast PWM and Non-Inverting output, **TCCR2B** for the prescaler, and the **OCR2x** registers for the specific duty cycles.

```
#include <avr/io.h>
void setup() {
  DDRB |= (1 << DDB3);
  DDRD |= (1 << DDD3);
  TCCR2A = (1 << WGM21) | (1 << WGM20) |
           (1 << COM2A1) | (1 << COM2B1);
  TCCR2B = (1 << CS22);
  OCR2A = 191;
  OCR2B = 63;
}
void loop() {
}
```

4. Theory of Operation.

1. Setting Fast PWM Mode:

- TCCR2A = (1 << WGM21) | (1 << WGM20)
- **Explanation:** These specific bits select **Mode 3 (Fast PWM)**. In this mode, the timer counts repeatedly from 0 to 255. It resets immediately upon hitting the top (0xFF), creating a sawtooth waveform that is ideal for controlling power delivery to LEDs.

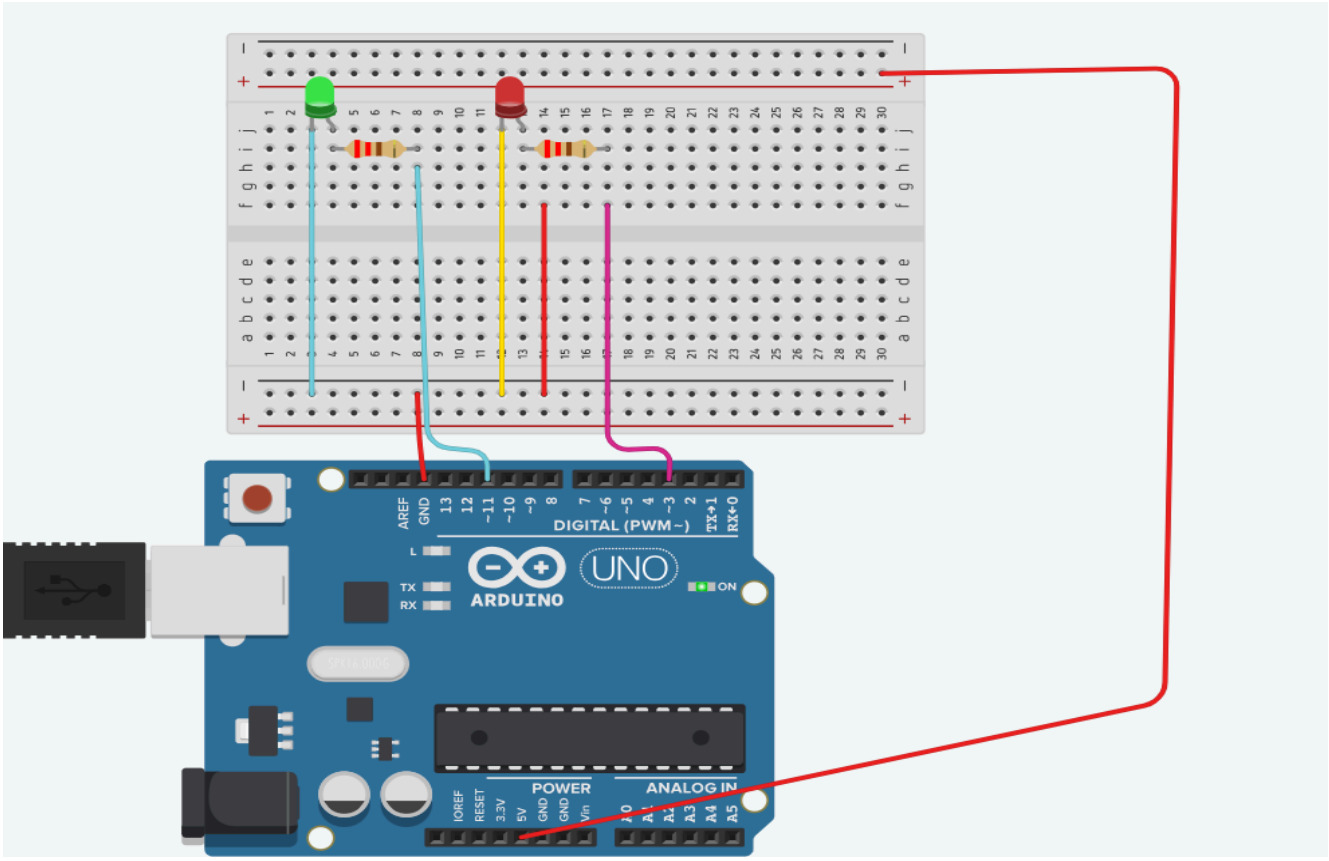
2. Configuring Non-Inverting Output:

- TCCR2A |= (1 << COM2A1) | (1 << COM2B1);
- **Explanation:** These bits control how the pins behave when the timer matches the compare value.
    - **Logic:** When the counter matches OCR2A, Pin 11 turns **LOW** (Off). When the counter resets to 0, Pin 11 turns **HIGH** (On).
    - This "Non-Inverting" behavior ensures that a higher number in the register corresponds to a brighter LED (higher duty cycle).

3. Setting the Duty Cycles:

- OCR2A = 191; // 75%
- OCR2B = 63; // 25%
- **Explanation:** The brightness is determined by how long the signal stays HIGH compared to the full count (255).
    - For **LED A**, we set OCR2A to 191 (approx 75% of 255). This means the LED is ON for the first 75% of the cycle and OFF for the remaining 25%.
    - For **LED B**, we set OCR2B to 63 (approx 25% of 255). This means the LED is ON for only 25% of the cycle, appearing much dimmer .

5. Simulation The custom driver relies on precise pin definitions to match the wiring in the simulation and physical build.



Upon running the simulation, **LED A (Pin 11)** appeared significantly brighter than **LED B (Pin 3)**. Although the LEDs are technically turning on and off 976 times per second (Prescaler 64), the human eye (and the simulator) perceives this as a steady brightness level due to the persistence of vision. This confirms that the PWM signals are being generated correctly with the distinct duty cycles of 75% and 25%.

V. Question 5: Analog to Digital Conversion (Register Level)

2. **Circuit Design**
   The goal of this task is to configure the **Analog-to-Digital Converter (ADC)** manually to create a dimmer switch. The program must read the varying voltage from a potentiometer connected to the ADC, convert it into a digital value, and map that result to control the brightness of an LED .

   - **TinkerCad Simulation Link:** question 4

2. Hardware Configuration

The circuit involves a potentiometer for analog input and an LED for PWM output.

Component	Arduino Pin	ATmega Port	Connection Detail
Potentiometer	Analog Pin A0	ADC0	Left leg to 5V; Right leg to GND; Middle (Wiper) to A0.
LED	Digital Pin 5	PD5	Anode to Pin 5; Cathode to GND via 220ohm resistor.

3. Software Implementation

```
#include <avr/io.h>

void setup() {

  pinMode(5, OUTPUT);


  ADMUX = (1 << REFS0);

  ADCSRA = (1 << ADEN) | (1 << ADPS2)
    | (1 << ADPS1) | (1 << ADPS0);
}

void loop() {

  ADCSRA |= (1 << ADSC);


  while (ADCSRA & (1 << ADSC));

  int adcValue = ADCL;
  adcValue |= (ADCH << 8);

  int pwmValue = adcValue >> 2;

  analogWrite(5, pwmValue);

  delay(10);
}
```

4. Theory of Operation.

This implementation manually drives the conversion process step-by-step rather than using analogRead().

1. Configuring the Reference & Clock:

C

- ADCSRA = (1 << ADEN) | (1 << ADPS2) | (1 << ADPS1) | (1 << ADPS0);

- **Explanation:** The ADC needs a clock frequency between 50kHz and 200kHz to be accurate. The Arduino runs at 16MHz. By setting the Prescaler bits (ADPS2:0) to 128, we reduce the clock to 125 kH, which is perfectly within the safe range .

2. Polling for Completion:

```
while (ADCSRA & (1 << ADSC));
```

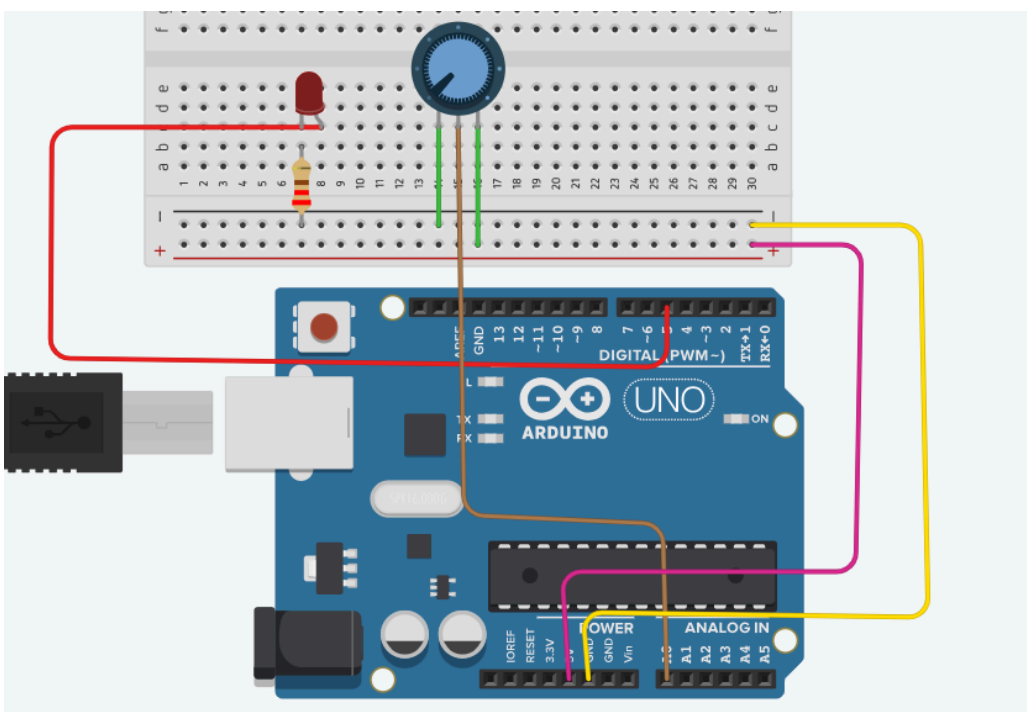
- **Explanation:** Unlike an interrupt, "Polling" involves constantly checking a status flag. When we set the ADSC (Start Conversion) bit, the hardware begins measuring the voltage. We force the processor to sit in this while loop as long as ADSC remains 1. Once the hardware finishes the measurement, it automatically flips ADSC to 0, breaking the loop and allowing the code to proceed.

3. Bit Mapping (Scaling):

- int pwmValue = adcValue >> 2;

- **Explanation:** The ADC provides a **10-bit resolution** (values 0 to 1023). However, the PWM output (controlled by analogWrite) only accepts **8-bit resolution** (values 0 to 255).
- Mathematically, 1024 / 4 = 256. In binary, shifting a number to the right by 2 positions (>> 2) is the same as dividing by 4, efficiently mapping the input range to the output range

5. Simulation



Upon running the simulation, rotating the potentiometer knob resulted in immediate changes to

the LED's brightness. When the potentiometer was set to 0V (min), the LED was off. As the dial was turned toward 5V (max), the LED smoothly transitioned from dim to full brightness. This confirms the correct operation of the ADC configuration, polling loop, and value mapping logic.

Conclusion

In conclusion, this assignment successfully demonstrated the fundamental principles of embedded systems programming by transitioning from high-level Arduino functions to direct register-level manipulation on the ATmega328P microcontroller. Through the implementation of five distinct tasks, we explored the critical hardware peripherals—Interrupts, Timers, PWM, and ADC—gaining deeper insight into how the microcontroller operates at the silicon level.

Key achievements include:

- **Efficient Event Handling:** By replacing polling loops with **External and Pin Change Interrupts**, we created a responsive system that reacts immediately to user input without wasting CPU cycles.
- **Precise Timing:** We utilized both **16-bit (Timer1)** and **8-bit (Timer2)** timers in CTC and Overflow modes to generate accurate time delays and events, eliminating the blocking nature of the `delay()` function.
- **Signal Generation & Interfacing:** We manually configured **Fast PWM** to control signal duty cycles for LED dimming and directly manipulated **ADC registers** to interface with analog sensors, mapping real-world voltage inputs to digital outputs.

All circuits and code were verified through simulation, confirming that direct access to registers like `EICRA`, `TCCR1B`, and `ADMUX` provides superior precision and control compared to standard abstraction layers. This exercise established a strong foundation for developing efficient, real-time embedded applications.